

Instruções da Prova

1. Esta prova é individual, teórica e prática, devendo ser realizada nos computadores do IMT, exclusivamente;
2. Não poderá haver acesso à Internet, sob nenhuma circunstância, exceto ao CANVAS LMS do próprio aluno e, mesmo assim, somente em duas etapas: para receber (“baixar”) as questões de sua prova na máquina em uso; e para entregar (“subir”) as resoluções das questões de sua prova no devido local de entrega, na mesma plataforma.
IMPORTANTE: O aluno deverá informar ao professor quando fará os dois acessos permitidos a ele ao CANVAS LMS, pela Internet;
3. Poderá haver consulta a qualquer material do próprio aluno, seja ele físico (livros, artigos, material de aula etc.) ou virtual (livros, artigos, material de aula, exercícios, resoluções etc.), desde que esse material esteja previamente armazenado em um dispositivo de armazenamento externo (*pendrive*, *hd* externo etc.) seu.
IMPORTANTE: Não será permitido o acesso pela Internet a pastas compartilhadas (*Google Drive*, *OneDrive* etc.), nem a repositórios virtuais (*GitHub*, *GitLab*, *BitBucket* etc.), mesmo sendo de posse e administração do próprio aluno;
4. O aluno deverá responder às questões da prova no próprio arquivo da prova (*.docx*) devidamente identificado (RA e Nome completo do Aluno) e quando for o caso, gerando os códigos necessários, somente na linguagem de programação JAVA e utilizando sempre e somente o paradigma da Programação Orientada a Objetos visto nas aulas;
5. Para realizar a entrega da prova na plataforma CANVAS LMS, o aluno deverá gerar e entregar um único arquivo compactado (*.zip*), tendo seu RA e seu NOME completo como nome desse arquivo. Nesse arquivo compactado o aluno deverá fornecer, obrigatoriamente, os seus dados acadêmicos preenchidos na prova, bem como a resolução das questões nela solicitadas (*.docx*), além dos arquivos e códigos gerados em suas resoluções, uma pasta para cada questão, contendo as suas classes e demais arquivos que possibilitem sua posterior execução pelo professor, durante a resolução.
IMPORTANTE: Na correção, para executar o código gerado pelo aluno em sua prova, o professor seguirá exatamente as instruções fornecidas pelos alunos e contidas nas resoluções das provas! Caso não obtenha sucesso, a questão será considerada errada.
6. Não poderá haver troca de informações, nem de materiais, sejam físicos ou virtuais, entre os alunos durante a prova;
7. Não é permitido ao estudante se ausentar da sala antes da entrega da prova;
8. Celulares, *Smartphones*, *Smartwatches*, *Smartglasses* ou outros equipamentos eletrônicos devem permanecer desligados enquanto o estudante estiver na sala;
9. O tempo limite para realização da prova é de **90** minutos;
10. Mantenha sobre a carteira apenas um documento com foto, caneta, lápis e borracha;
11. O entendimento das questões faz parte da avaliação;
12. O tempo mínimo de permanência na sala é de **30** minutos;
13. O estudante que chegar atrasado em até **30** minutos do início da prova poderá fazê-la.

QUESTÕES

Q1 – 1,5 PONTO:

Considere o *heap* máximo representado e implementado, a partir do índice 0, pelo vetor:

[90, 70, 80, 40, 60, 50, 65]

A chave 85 é inserida. Logo a seguir, o maior elemento do *heap* é removido. Todas as operações de reorganização devem seguir o algoritmo estudado em aula.

Qual vetor representa corretamente o *heap* resultante?

- A) [85, 80, 70, 40, 60, 50, 65]
- B) [85, 70, 80, 40, 60, 50, 65]
- C) [80, 70, 65, 40, 60, 50, 85]
- D) [90, 70, 80, 40, 60, 50, 65]
- E) [85, 70, 80, 60, 40, 50, 65]

Q2 – 1,5 PONTO:

Uma tabela *hash* possui sete posições, numeradas de 0 a 6. Ela utiliza:

$$h(k) = k \bmod 7$$

As colisões são resolvidas por *rehashing* com busca linear circular. Em uma tabela inicialmente vazia, são inseridas, nesta ordem, as chaves:

10, 24, 31, 18 e 17

Após as inserções, realiza-se a busca pela chave 17.

Assinale a alternativa que apresenta corretamente a tabela e a sequência de posições examinadas na busca.

- A) Tabela [17, -, -, 10, 24, 31, 18]; busca 3, 4, 5, 6, 0.
- B) Tabela [18, 17, -, 10, 24, 31, -]; busca 3, 4, 5, 6, 0, 1.
- C) Tabela [17, -, -, 10, 31, 24, 18]; busca 3, 4, 5, 6, 0.
- D) Tabela [-, 17, -, 10, 24, 31, 18]; busca 3, 4, 5, 6, 0, 1.
- E) Tabela [17, -, -, 10, 24, 18, 31]; busca 3, 4, 5, 6, 0.

Q3 1,5 PONTO:

Considere o grafo simples, não direcionado:

$$V = \{A, B, C, D, E, F\} \quad E =$$

{AB, BC, CD, DA, AC, CE, EF, FC} Analise as afirmações:

I. A sequência não crescente dos graus é (5, 3, 2, 2, 2, 2).

-

- II. O grafo é *Euleriano* porque possui exatamente dois vértices de grau ímpar.
- III. A sequência A-B-C-D-A-C-E-F-C constitui uma *Trilha Euleriana* aberta.
- IV. A sequência D-A-B-C-E-F constitui um *Caminho Hamiltoniano*.
- V. O grafo é *Hamiltoniano* porque possui um caminho que contém todos os seus vértices.

Assinale a única alternativa correta:

- a) Apenas I, III e IV são verdadeiras.
- b) Apenas I, II e IV são verdadeiras.
- c) Apenas II, III e V são verdadeiras.
- d) Apenas I, III, IV e V são verdadeiras.
- e) Todas as afirmações são verdadeiras.

-

Q4 1,5 PONTO:

Considere o grafo ciclo:

$C_6 = A-B-C-D-E-F-A$ Esse

grafo possui seis vértices, todos com grau 2.

Um estudante argumentou:

“Como cada vértice possui grau 2, o grafo não satisfaz a condição do *Teorema de Dirac*, que exigiria grau mínimo 3. Logo, o grafo não pode ser *Hamiltoniano*.”

Assinale a análise correta.

- a) O estudante está correto, pois *Dirac* estabelece uma condição necessária e suficiente.
- b) O estudante está correto, pois todo grafo hamiltoniano com seis vértices deve ter grau mínimo 3.
- c) O estudante está incorreto: o próprio ciclo apresentado é *hamiltoniano*, e *Dirac* fornece uma condição suficiente, mas não necessária.
- d) O estudante está incorreto porque o grafo satisfaz *Dirac*, uma vez que todos os vértices possuem o mesmo grau.
- e) O estudante está correto, mas apenas porque o grafo também deixa de satisfazer o *Teorema de Ore*.

Q5 4,0 PONTOS:

Uma empresa precisa manter, em memória, um catálogo reduzido de componentes eletrônicos. Cada componente possui:

- Código: número inteiro positivo;
- Descrição: Texto.

Implemente, em *Java* e sob o paradigma da Programação Orientada a Objetos, uma tabela *hash* que armazene objetos da classe *Componente*.

Requisitos

A. Classe *Componente* Deverá armazenar:

```
private int codigo; private  
String descricao;
```

-

Implemente construtor, métodos necessários e uma representação textual adequada.

B. Classe *TabelaHash*

Deverá possuir internamente:

```
private Componente[] tabela;
```

A tabela deverá ter exatamente 11 posições, numeradas de 0 a 10.

A função *hash* deverá ser:

```
h(codigo) = codigo mod 11
```

Não poderão ser utilizadas *HashMap*, *Hashtable*, coleções prontas ou outra implementação automática de tabela *hash*. Somente códigos manuais!

C. Inserção e colisões

Implemente um método para inserir componentes.

Quando houver colisão, utilize *rehashing* com busca linear circular, examinando sucessivamente: $(h(\text{codigo}) + i) \bmod 11$ para $i = 0, 1, 2, \dots$

A inserção deverá:

- Armazenar o componente na primeira posição livre encontrada;
- Impedir códigos duplicados;
- Informar quando a tabela estiver cheia;
- Nunca acessar uma posição fora dos limites do vetor.

D. Busca

Implemente um método que receba um código e:

- Retorne o componente correspondente, quando encontrado;
- Retorne *null*, quando ele não estiver armazenado;
- Encerre a busca ao encontrar uma posição vazia ou após examinar toda a tabela.

E. Apresentação

Implemente um método que apresente todas as posições da tabela, inclusive as posições vazias.

F. Classe de execução

Crie uma classe de execução que realize, obrigatoriamente, nesta ordem:

```
Inserir (22, "Resistor")
```

Inserir (13, "Capacitor")
Inserir (35, "Diodo")
Inserir (24, "Transistor")
Inserir (46, "Sensor")
Inserir (57, "Microcontrolador")
Buscar 46
Inserir novamente o código 35
Buscar 99
Exibir toda a tabela

A aplicação deverá produzir resultados que permitam verificar claramente:

- A posição final de cada componente;
- O sucesso da busca por 46;
- A rejeição do código duplicado 35;
- O resultado negativo da busca por 99.

G. Entrega:

- Todos os códigos-fonte; ○ Arquivo *.jar* executável; ○ Instruções claras e precisas para a execução.

RUBRICA DA Q5

<u>Critério</u>	<u>Valor</u>
Classe Componente corretamente encapsulada.....	0,40
Estrutura da classe <i>TabelaHash</i>	0,40
Função <i>hash</i> pelo método da divisão.....	0,40
Inserção sem colisão.....	0,40
<i>Rehashing</i> linear circular.....	0,80
Detecção de código duplicado e tabela cheia.....	0,40
Busca correta e com condição de parada.....	0,60
Exibição da tabela e execução dos casos solicitados:	0,40
Arquivo <i>.jar</i> funcional e entrega completa	0,20
TOTAL.....	4,00